

BINOMIAL MIN HEAP

VIVA QUESTIONS AND ANSWERS

CSE 208 - Data Structures and Algorithms II Sessional

Student	Lamia Buruz
Student ID	2405118
Coverage	Part A implementation, Part B visualizer, complexities, correctness, and viva traps

How to use this guide

Practice answering each question aloud in 15-30 seconds. For implementation questions, point to the relevant function and explain how it preserves the three invariants: heap order, increasing root degree, and one tree per degree. Answers are deliberately concise so they can be used directly in the viva.

A. Core Concepts

Q1. What is a heap?

Answer: A heap is a tree-based data structure that obeys a heap-order property. In a min heap, every parent key is less than or equal to the keys of its children.

Q2. What is a binomial heap?

Answer: A binomial heap is a collection of heap-ordered binomial trees with at most one tree of each degree.

Q3. Why is it called a binomial heap?

Answer: The number of nodes at depth i in a binomial tree B_k is the binomial coefficient $C(k, i)$.

Q4. What is a binomial tree B_0 ?

Answer: B_0 is a single node. It has one node, height zero, and root degree zero.

Q5. How is B_k constructed?

Answer: B_k is constructed by linking two B_{k-1} trees and making one root the first child of the other root.

Q6. How many nodes are in B_k ?

Answer: B_k contains 2^k nodes.

Q7. What is the height of B_k ?

Answer: The height of B_k is k .

Q8. What is the degree of the root of B_k ?

Answer: The root degree is k .

Q9. How many nodes are at depth i in B_k ?

Answer: There are $C(k,i)$ nodes at depth i .

Q10. What is the min-heap-order property?

Answer: For every parent-child edge, the parent key is less than or equal to the child key.

Q11. Where can the minimum key be found in a binomial min heap?

Answer: It must be among the roots because no child can be smaller than its parent.

Q12. Can two trees of the same degree remain in a valid binomial heap?

Answer: No. A valid binomial heap contains at most one tree of each degree.

Q13. How many trees can a binomial heap with n nodes contain?

Answer: At most $\text{floor}(\log_2 n) + 1$ trees.

Q14. How is heap size related to binary representation?

Answer: A set bit at position k in the binary form of n means that the heap contains a B_k tree.

Q15. If a heap has 13 nodes, which tree orders can it contain?

Answer: 13 is 1101 in binary, so it contains B_0 , B_2 , and B_3 .

Q16. If a heap contains B_1 and B_4 , what is its size?

Answer: $2^1 + 2^4 = 2 + 16 = 18$ nodes.

Q17. Is a binomial heap one complete tree?

Answer: Not necessarily. It is a forest, meaning a collection of binomial trees.

Q18. What makes binomial heaps useful?

Answer: Their root lists can be merged efficiently, so Union is much faster than in many ordinary heap representations.

Q19. What is the difference between node degree and tree order?

Answer: Node degree is its number of direct children. For a binomial tree root, its degree also equals the order of that tree.

Q20. Does degree count grandchildren?

Answer: No. It counts only direct children.

B. Node and Class Design

Q21. What does the Node class represent?

Answer: It represents one stored heap element and keeps its key, degree, parent pointer, and child pointers.

Q22. Why is key an integer?

Answer: The assignment specifies integer keys in the range -10^9 to 10^9 .

Q23. Why does a new node start with degree zero?

Answer: A newly inserted node initially has no children and forms a B0 tree.

Q24. Why is the parent pointer initially NULL?

Answer: The new node begins as a root, and roots have no parent.

Q25. Why use vector children?

Answer: It provides supporting storage for all direct child pointers while preserving the actual tree relationships.

Q26. Does using vector violate the assignment?

Answer: No. Ordinary containers are allowed as supporting storage. The code still implements actual binomial trees and does not replace them with a built-in heap.

Q27. What does roots store?

Answer: It stores pointers to the roots of all binomial trees in the heap.

Q28. In what order are roots stored?

Answer: They are stored in increasing order of degree.

Q29. What does heapSize store?

Answer: It stores the total number of nodes across every tree, not the number of roots.

Q30. Why are roots and heapSize private?

Answer: Encapsulation prevents outside code from changing the internal heap structure without using heap operations.

Q31. Why are operations public?

Answer: The caller must be able to insert, search, union, print, and visualize while the representation remains protected.

Q32. Where is each node's degree stored?

Answer: It is stored in the Node member variable named degree.

Q33. Where does degree begin?

Answer: The Node constructor sets degree to zero.

Q34. Where is degree increased?

Answer: linkTrees increments the degree of the root that becomes the parent.

Q35. Where is the degree printed beside a node?

Answer: showTree prints node->degree beside node->key.

C. Linking, Merging, and Union

Q36. What does `linkTrees` do?

Answer: It links two trees of the same degree, makes the smaller root the parent, adds the other root as its first child, increments the parent degree, and returns the new root.

Q37. Why must the trees passed to `linkTrees` have equal degrees?

Answer: Only two B_k trees can be combined to form one valid B_{k+1} tree.

Q38. Why does the smaller root become the parent?

Answer: This preserves the min-heap-order property.

Q39. Why is the losing root inserted as the first child?

Answer: The assignment specifies this convention so that the exact structure is reproducible.

Q40. Does `linkTrees` physically swap nodes?

Answer: No. It changes parent and child pointers. Key swapping is used only during decrease-key.

Q41. What are the two main stages of Union?

Answer: First merge the two root lists by degree. Then consolidate equal-degree neighboring trees from left to right.

Q42. How is merging similar to merge sort?

Answer: Two indices scan already ordered root lists and repeatedly append the root with the smaller degree.

Q43. What may exist immediately after merging root lists?

Answer: Multiple neighboring roots can have the same degree, so consolidation is still required.

Q44. What does `differentDegrees` check?

Answer: It checks whether the current root and the next root have unequal degrees. If so, they cannot be linked.

Q45. What does `threeSameDegrees` check?

Answer: It checks whether the current root and the root two positions ahead have the same degree, which means three consecutive roots share that degree.

Q46. Why skip the first of three equal-degree roots?

Answer: Linking the first two would create a higher-degree tree before the third lower-degree tree and break degree order. The algorithm advances and links the second and third.

Q47. Why is `i` not increased immediately after a link?

Answer: The newly formed tree may have the same degree as the next root, so the same position must be checked again.

Q48. What happens to the second heap after Union?

Answer: Its root list is cleared and its size becomes zero because its trees have been transferred to the first heap.

Q49. Are nodes deleted when the second heap is cleared?

Answer: No. Their pointers have already been placed in the first heap, so the nodes remain alive.

Q50. What does `U 1 2` mean?

Answer: It stores `Union(H1,H2)` in `H1` and makes `H2` empty.

Q51. What is the time complexity of Union?

Answer: $O(\log n)$, because each heap has only $O(\log n)$ roots.

Q52. Can Union be implemented by reinserting all keys?

Answer: No. The assignment forbids rebuilding the heap from all stored keys.

Q53. Why is Union the central binomial heap operation?

Answer: Insert is performed by union with a one-node heap, and extract-min unions the removed root's child heap with the remaining heap.

Q54. What happens to heapSize in Union?

Answer: The other heap's size is added to the current heap, and the other heap's size is reset to zero.

Q55. How does the code ensure at most one tree of each degree?

Answer: The consolidation loop repeatedly links eligible equal-degree neighboring roots until no duplicate degree remains.

D. Insert, Find Min, and Extract Min

Q56. How does insertKey work?

Answer: It creates a temporary one-node heap containing a B0 tree and unions that heap with the current heap.

Q57. Why not simply append a new root?

Answer: Appending could leave two B0 trees. Union performs the required carry-like consolidation.

Q58. How is insertion like binary addition?

Answer: Adding a B0 may cause B0+B0 to become B1, then B1+B1 to become B2, like carries in binary addition.

Q59. What is insertion complexity?

Answer: $O(\log n)$ in the worst case because a sequence of links may travel across the root list.

Q60. How does findMin work?

Answer: It scans every root and returns the smallest root key.

Q61. Why does findMin not search children?

Answer: Heap order guarantees that every child's key is at least its root ancestor's key.

Q62. What is findMin complexity in this implementation?

Answer: $O(\log n)$, since the number of roots is $O(\log n)$.

Q63. What is the first task in extractMin?

Answer: Find the index of the root containing the minimum key.

Q64. What happens after the minimum root is removed?

Answer: Its children become roots of a temporary child heap.

Q65. Why are the minimum root's children reversed?

Answer: New children were inserted at the front, so they are stored from higher degree to lower degree. The root list must be lower degree to higher degree.

Q66. Why are child parent pointers set to NULL?

Answer: The children become roots of independent trees in the temporary heap.

Q67. How many descendants remain after removing the root of Bk?

Answer: $2^k - 1$ nodes remain.

Q68. What does $1 \ll \text{degree}$ calculate?

Answer: For the relevant nonnegative degree, it calculates 2^{degree} using a left bit shift.

Q69. Why clear minimum->children before deleting minimum?

Answer: The children have been transferred to childHeap. Clearing documents that the deleted node no longer owns those pointers.

Q70. Why call Union at the end of extractMin?

Answer: The child trees must be combined with the remaining root list and duplicate degrees must be consolidated.

Q71. By how much does extractMin change the final heap size?

Answer: It decreases the total size by exactly one.

Q72. What is extractMin complexity?

Answer: $O(\log n)$: it scans roots, processes $O(\log n)$ child roots, and performs Union.

Q73. What assumption makes empty-heap checks unnecessary in the submitted tests?

Answer: The assignment guarantees that Find Min and Extract Min will not be called on an empty heap.

E. Search, Decrease Key, and Remove

Q74. How does findInTree search?

Answer: It uses recursive depth-first search: check the current node, then recursively search each child subtree.

Q75. How does findNode search the whole heap?

Answer: It calls findInTree on every binomial-tree root until a match is found.

Q76. What is the search complexity?

Answer: $O(n)$ in the worst case because the code may inspect every node.

Q77. Does that violate the stated decrease-key bound?

Answer: No. The assignment says the $O(\log n)$ bound applies after the node has been located, matching the handle or pointer assumption used in class.

Q78. How does decreaseKey work after locating the node?

Answer: It assigns the smaller key, then repeatedly exchanges key values with the parent while the current key is smaller.

Q79. Why swap key values instead of node pointers?

Answer: The assignment explicitly requires key exchanges so that the binomial-tree structure remains unchanged.

Q80. When does bubbling stop?

Answer: It stops when the node becomes a root or its key is no longer smaller than its parent's key.

Q81. Why must y be smaller than x ?

Answer: This operation only repairs upward violations. Increasing a key could violate heap order with children and requires a different operation.

Q82. What assumptions are made about decrease-key input?

Answer: x exists, y is smaller than x , and y does not exist in either heap.

Q83. How is removeKey implemented?

Answer: It decreases the chosen key to a sentinel smaller than every valid input key and then calls extractMin.

Q84. Why does the sentinel move to a root?

Answer: Because it is smaller than every valid key, repeated key exchanges move it upward until it reaches a root.

Q85. Why not use -10^9 as the sentinel?

Answer: -10^9 is a valid input key and may already exist, so the sentinel must be strictly smaller.

Q86. Why is -1000000001 valid here?

Answer: It equals $-10^9 - 1$, so it is strictly smaller than the minimum allowed input key and still fits in a 32-bit signed int.

Q87. What happens if duplicate input keys exist?

Answer: Search by value would become ambiguous, but the assignment guarantees unique keys across both heaps.

Q88. What is remove complexity after locating the node?

Answer: $O(\log n)$: bubble the sentinel upward and extract the minimum.

F. Printing, Visualization, and File I/O

Q89. Why does printHeap use a queue?

Answer: A queue supports breadth-first traversal, which prints the tree level by level as required.

Q90. What does nodesThisLevel represent?

Answer: It records how many queued nodes belong to the current level before their children are added.

Q91. Why are keys sorted within each level?

Answer: The assignment's fixed print format requires increasing numerical order within a level.

Q92. Does sorting keys alter the heap?

Answer: No. Only a temporary vector of integer copies is sorted.

Q93. Why can standard P output not show exact parent-child relationships?

Answer: It groups and sorts keys by level, so their actual left-to-right parent assignments are not preserved in that output.

Q94. What is the role of showTree?

Answer: It recursively prints actual node connections using indentation and branch markers.

Q95. What do prefix and last control?

Answer: prefix stores indentation and continuation lines; last selects the final-child branch marker.

Q96. What is displayed by V h?

Answer: Every tree is shown separately with its order, root, actual parent-child structure, and each node's degree.

Q97. What is displayed by W h1 h2?

Answer: Both heaps before Union, every link in order, and the final first heap after Union.

Q98. What is the student's own useful feature?

Answer: The visualizer prints the degree beside every individual node, helping the evaluator verify direct-child counts and structural changes.

Q99. Which exact code prints the feature?

Answer: The showTree expression that appends '(degree ' followed by node->degree and ')' beside node->key.

Q100. Why is Part B output terminal-only?

Answer: It prevents creative visualization lines from changing the exact Part A output.txt used by automatic checking.

Q101. How are actual structures reused in Part B?

Answer: visualize and showTree directly traverse the same roots and child pointers maintained by Part A.

Q102. Does this visualizer use Graphviz?

Answer: No. It computes indentation itself with recursive prefix strings.

Q103. What does ostream& out provide?

Answer: It makes a printing function reusable with cout, an output file, or another output stream.

Q104. What does writeLine do?

Answer: It writes the same Find Min or Extract Min line to both cout and output.txt.

Q105. Why is P called twice?

Answer: The same printHeap function is called once with cout and once with the output file so both Part A outputs are identical.

Q106. Which commands produce normal Part A output?

Answer: Only F, E, and P. Insert, decrease, remove, and ordinary union produce no normal output.

Q107. What exact spacing correction is required in the tree heading?

Answer: It must print 'Binomial Tree, Bk' with a space after the comma, not 'Binomial Tree,Bk'.

Q108. What happens when output.txt is opened with ofstream?

Answer: Its previous contents are normally truncated, and the current run writes a fresh output file.

G. Complexity, Correctness, and Viva Traps

Q109. What are the core invariants of the heap?

Answer: Every tree is min-heap-ordered, roots are ordered by increasing degree, and no two roots have the same degree after consolidation.

Q110. How would you prove linkTrees is correct?

Answer: The input trees retain their internal structure; the smaller root becomes parent, preserving heap order across the new edge; and two B_k trees form one B_{k+1} .

Q111. How would you prove Union is correct?

Answer: The merge preserves degree order. Consolidation follows the prescribed cases, links only equal-degree trees, preserves heap order, and finishes with unique root degrees.

Q112. Why is the root count logarithmic?

Answer: Each existing degree represents at least 2^k nodes, and degrees correspond to set bits of the heap size.

Q113. What is the difference between merge and Union?

Answer: Merge only combines ordered root lists. Union includes merge plus consolidation of equal-degree trees.

Q114. What would happen if threeSameDegrees were removed?

Answer: With three equal-degree roots, the code could link the first two and disrupt increasing degree order, leading to incorrect consolidation.

Q115. What would happen if children were not reversed in extractMin?

Answer: The child heap roots would be in decreasing degree order, violating the assumption used by the merge stage.

Q116. What would happen if degree were not incremented after linking?

Answer: The resulting tree would be labeled with the wrong order and future consolidation decisions would be incorrect.

Q117. What would happen if the second heap were not emptied after Union?

Answer: Both heap objects would refer to the same nodes, violating the required semantics and risking later structural corruption.

Q118. Why is a built-in priority queue unsuitable?

Answer: It does not expose the required binomial-tree forest, link sequence, or actual structures needed by Part B, and the assignment explicitly forbids it.

Q119. Is the code's heap numbering safe for valid input?

Answer: Yes. BinomialHeap heaps[3] provides indices 0, 1, and 2; the assignment uses H1 and H2, so indices 1 and 2 are valid.

Q120. Why is heaps[0] present?

Answer: It allows the program to use the assignment's heap numbers directly without subtracting one. Index zero is unused.

Q121. What limitation does the current class have regarding memory?

Answer: It has no destructor to recursively delete nodes left in the heap at program termination. The operating system reclaims process memory, but a reusable class should implement ownership carefully.

Q122. Why would adding a destructor require care?

Answer: Union transfers raw node pointers between heap objects. Incorrect copying or destruction could cause double deletion, so copy and move behavior must also be designed.

Q123. What exact input assumptions protect decreaseKey and removeKey?

Answer: The assignment guarantees that target keys exist, decreased values are smaller and unique, and all commands are otherwise valid.

Q124. What should you say if asked whether the program is perfect for arbitrary input?

Answer: It is designed for the assignment's valid-input assumptions. A production version would add validation for empty heaps, missing keys, invalid heap indices, and invalid decreases.

Q125. What is the maximum displayed tree in Part B evaluation?

Answer: The assignment states that no displayed tree will be larger than B4 and there will be at most 31 elements in total.

Q126. Why is the recursive visualizer acceptable for B4?

Answer: B4 has only 16 nodes, so recursive traversal and indentation are small and clear.

Q127. What is the strongest short description of the whole program?

Answer: It maintains two actual binomial min heaps, implements operations through root-list merging and tree linking, produces exact Part A output, and visualizes the same structures and link sequence for Part B.

Q128. What three functions should you be most ready to explain?

Answer: unionWith, extractMin, and linkTrees, because they establish and preserve the central binomial-heap structure.

Last-Minute Rapid Revision

Invariant	Heap ordered; roots increase by degree; no duplicate root degree
Bk facts	2^k nodes, height k, root degree k
Union	Merge by degree, then consolidate left to right
Three equal degrees	Skip the first; consider the second and third
Insert	Union with a one-node B0 heap
Find Min	Scan roots only
Extract Min	Remove minimum root, reverse children, union child heap
Decrease	Swap key values upward; do not swap physical nodes
Remove	Decrease below -10^9 , then Extract Min
Own feature	Degree displayed beside every individual node
Output warning	Print exactly: Binomial Tree, Bk

Total viva questions: 128